

Azure Data Analytics Case Study: Trip Data Analysis using PySpark on Azure Cloud

1. Introduction

Modern transportation systems generate large volumes of trip data, capturing travel times, distances, purposes, and locations. Analyzing this data can provide insights into travel behavior, operational efficiency, and usage patterns. Traditional data tools often struggle with large datasets, making big data technologies such as Apache Spark essential for scalable processing.

This case study demonstrates trip data analysis using **PySpark** on **Microsoft Azure**. The analysis aims to identify travel patterns, evaluate efficiency, detect anomalies, and derive actionable insights.

2. System Architecture

The workflow leverages Azure cloud infrastructure:

- **Data Storage:** Azure Data Lake Storage for scalable storage of raw datasets.
- **Data Processing:** Azure Databricks with PySpark for distributed processing, cleaning, and transformation.
- **Analysis:** Spark SQL and PySpark transformations produce aggregated results.
- **Visualization:** Python libraries **Matplotlib** and **Seaborn** create visual representations of insights.

This architecture supports scalable, efficient processing of large-scale datasets.

3. Azure Storage Account Set up

Creating a Storage Account named **tripdata**.

The screenshot shows the 'Create a storage account' page in the Azure portal. The page is titled 'Create a storage account' and has a close button (X) in the top right corner. The page is divided into two main sections: 'Project details' and 'Instance details'.

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription *

Resource group * [Create new](#)

Instance details

Storage account name *

Region * [Deploy to an Azure Extended Zone](#)

Preferred storage type

ⓘ This helps us provide relevant guidance. It doesn't restrict your storage to this

Navigation buttons: [Previous](#) [Next](#) [Review + create](#) [Give feedback](#)

Deployment

The screenshot shows the 'tripdata_1774930847054 | Overview' page in the Azure portal. The page is titled 'tripdata_1774930847054 | Overview' and has a close button (X) in the top right corner. The page is divided into two main sections: 'Overview' and 'Deployment details'.

Overview

Deployment

Search x «

Actions: [Delete](#) [Cancel](#) [Redeploy](#) [Download](#) [Refresh](#)

Deployment details

Next steps

[Go to resource](#)

Your deployment is complete

Deployment name : tripdata_1774930847054 Start time : 3/31/2026, 9:50:57 AM

Subscription : Learner Account Correlation ID : fcd5b916-88c6-461c-9df9-0b3...

Resource group : rg-azuser6172_mml.local-XDIPu

Cost management

Get notified to stay within your budget and prevent unexpected charges on your bill. [Set up cost alerts >](#)

Microsoft Defender for Cloud

Secure your apps and infrastructure [Go to Microsoft Defender for Cloud >](#)

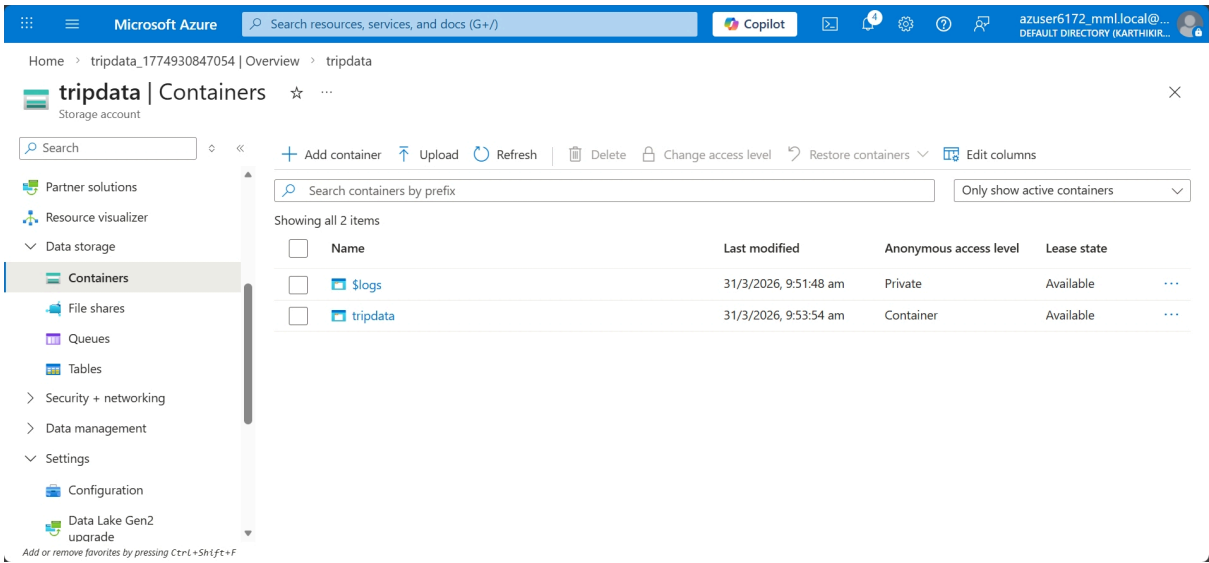
Free Microsoft tutorials

[Start learning today >](#)

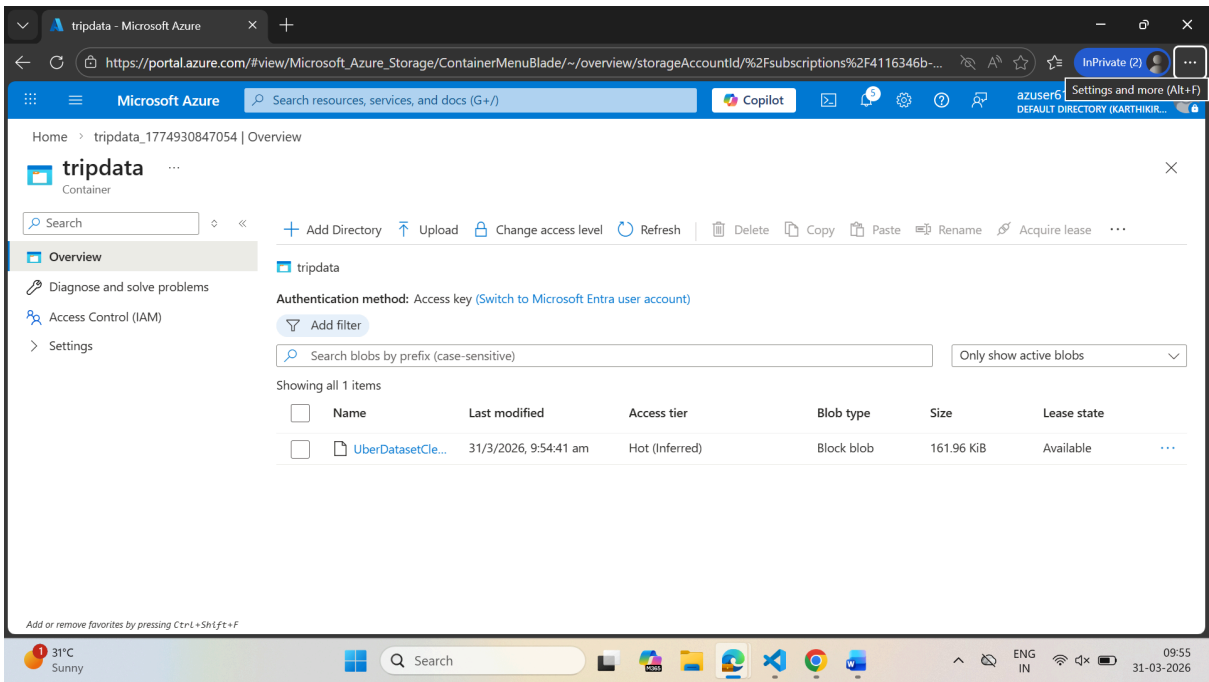
Work with an expert

Add or remove favorites by pressing Ctrl+Shift+F

Creating a container named “tripdata” in the storage account.



Adding the CSV file “UberDatasetCleaned” in the container.



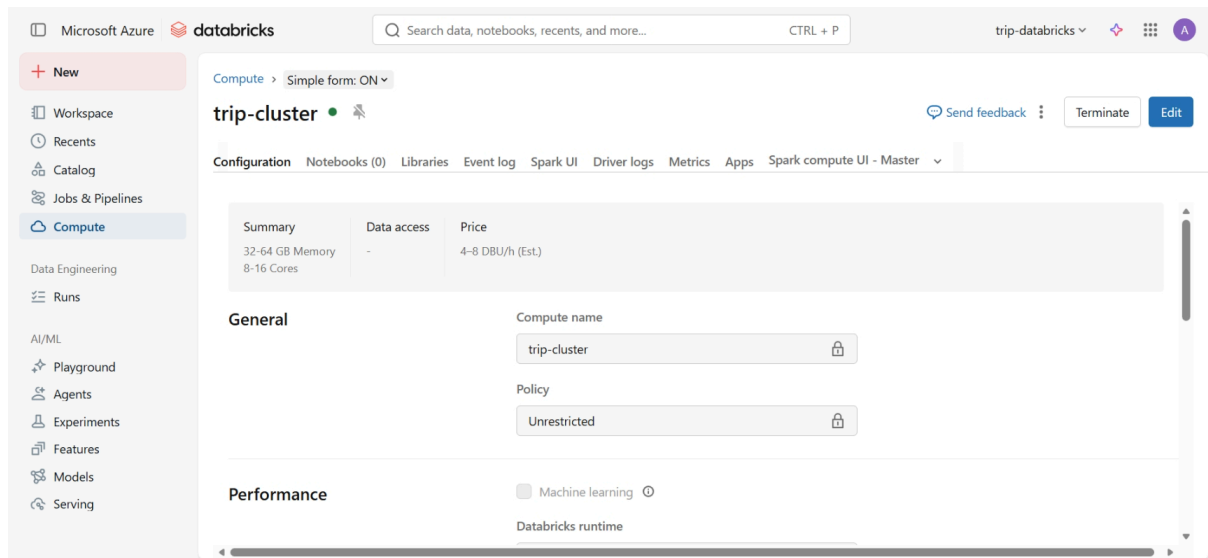
Creating a databrick name “trip-databricks” for data processing

The screenshot shows the Microsoft Azure portal interface. At the top, there's a navigation bar with the Microsoft Azure logo, a search bar, and a Copilot button. Below the navigation bar, the page title is "rg-azuser6172_mml.local-XDfPu_trip-databricks | Overview". The left sidebar contains a search bar and a list of navigation items: Overview (selected), Inputs, Outputs, and Template. The main content area displays a green checkmark icon and the text "Your deployment is complete". Below this, there are details about the deployment: Deployment name, Subscription, Resource group, Start time, and Correlation ID. There are also links for "Deployment details" and "Next steps", and a "Go to resource" button. On the right side, there are three promotional cards: "Cost management", "Microsoft Defender for Cloud", and "Free Microsoft tutorials".

After creating the databrick, click launch workspace, inside the page click **compute** to create a new cluster named “trip-cluster”

The screenshot shows the Databricks workspace interface. At the top, there's a navigation bar with the Microsoft Azure logo, the Databricks logo, a search bar, and a "CTRL + P" button. Below the navigation bar, the page title is "trip-databricks". The left sidebar contains a list of navigation items: New (selected), Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Data Engineering, Runs, AI/ML, Playground, Agents, Experiments, Features, Models, and Serving. The main content area displays the "Create new compute" form. The form has two tabs: "UI" (selected) and "JSON". The form is divided into two sections: "General" and "Performance". In the "General" section, there are fields for "Compute name" (set to "trip-cluster") and "Policy" (set to "Unrestricted"). In the "Performance" section, there are checkboxes for "Machine learning" and "Photon acceleration" (checked). There are also dropdown menus for "Databricks runtime" (set to "17.3 LTS") and "Preferred worker type" (set to "Standard_D4ds_v4"). There are input fields for "Min" (set to "2") and "Max" (set to "4") values, and a "Single node" checkbox. At the bottom of the form, there are "Create" and "Cancel" buttons.

Creating the cluster



After creating a cluster, create a notebook and select python for processing and analysing the data. Attach the notebook with a cluster and then start run.

```
Just now (<1s) 2 Python
```

```
# Connect databricks to storage
storage_account = "tripdata"
container = "tripdata"
access_key = "/iJdBjZZedcmcEwhJFzBpPtNfNvo1aLFFFn18eWLIj5JNo5jU7x/WXNGIU8mgN17GPcb3eZzU+YZ+ASTfmBNfA=="

spark.conf.set(
    f"fs.azure.account.key.{storage_account}.blob.core.windows.net",
    access_key
)
```

Connect the notebook with the storage container and set an SAS access token, so that the file can be accessed via databricks. After connecting, read the csv.

```
1 minute ago (11s) 3
```

```
# Read CSV
df = spark.read.csv(
    "wasbs://tripdata@tripdata.blob.core.windows.net/UberDatasetCleaned.csv",
    header=True,
    inferSchema=True
)

df.show(10)
```

(3) Spark Jobs

df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 15 more fields]

start_date	start_hour	start_minute	end_date	end_hour	end_minute	month	year	CATEGORY	START	STOP
2016-01-01	21.0	11.0	2016-01-01	21.0	17.0	Jan	2016	Business	Fort Pierce	Fort Pierce
2016-01-01	8.0	2076339999999999	2016-01-01	8.0	82.07633999999999	Friday	Evening			
2016-01-02	1.0	25.0	2016-01-02	1.0	37.0	Jan	2016	Business	Fort Pierce	Fort Pierce

Check the datatypes and clean, cast the data if need

▶ ✓ 3 minutes ago (<1s)

```
# Check Datatypes
df.printSchema()
```

```
root
|-- start_date: date (nullable = true)
|-- start_hour: double (nullable = true)
|-- start_minute: double (nullable = true)
|-- end_date: date (nullable = true)
|-- end_hour: double (nullable = true)
|-- end_minute: double (nullable = true)
|-- month: string (nullable = true)
|-- year: integer (nullable = true)
|-- CATEGORY: string (nullable = true)
|-- START: string (nullable = true)
|-- STOP: string (nullable = true)
|-- MILES: double (nullable = true)
|-- PURPOSE: string (nullable = true)
|-- duration: double (nullable = true)
|-- speed: string (nullable = true)
|-- day: string (nullable = true)
|-- day_time: string (nullable = true)
```

4. Analytical Use Cases

4.1 Trip Duration Analysis

Objective: Calculate average trip duration, identify unusually long or short trips, and compare across trip categories.

Finding the Average Trip Duration

▶ ✓ 1 minute ago (1s) 5

```
# Average Trip Duration
from pyspark.sql.functions import avg, round

df.select(
    round(avg("duration"), 2).alias("avg_trip_duration")
).show()
```

▶ (2) Spark Jobs

```
+-----+
|avg_trip_duration|
+-----+
|          22.68|
+-----+
```

Finding Long Duration Trips

```
▶ 1 minute ago (1s) 6 Python
```

```
# Long Duration Trips
long_trips = df.filter(df.duration > 60)
long_trips.show()
```

▶ (1) Spark Jobs

> long_trips: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 15 more fields]

start_date	start_hour	start_minute	end_date	end_hour	end_minute	month	year	CATEGORY	START	STO
P	MILES	PURPOSE	duration	speed	day	day_time				
2016-01-06	14.0	42.0	2016-01-06	15.0	49.0	Jan	2016	Business	Fort Pierce	West Palm Beac
h 102.51495800000001		Customer Visit	67.0	91.80444	Wednesday	Afternoon				
2016-02-16	8.0	29.0	2016-02-16	9.0	34.0	Feb	2016	Business	Unknown Location	Colomb
o 22.691694		Customer Visit	65.0	20.946179076923077	Tuesday	Morning				
2016-02-20	14.0	50.0	2016-02-20	15.0	54.0	Feb	2016	Business	R?walpindi	R?walpind
i 37.175754000000005		Meeting	64.0	34.852269375000006	Saturday	Afternoon				

Finding Short Duration Trips

```
▶ Just now (1s) 7
```

```
# Short Duration Trips
short_trips = df.filter(df.duration < 10)
short_trips.show()
```

▶ (1) Spark Jobs

> short_trips: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 15 more fields]

start_date	start_hour	start_minute	end_date	end_hour	end_minute	month	year	CATEGORY	START	STO
P	MILES	PURPOSE	duration	speed	day	day_time				
2016-01-01	21.0	11.0	2016-01-01	21.0	17.0	Jan	2016	Business	Fort Pierce	Fort Pierc
e 8.207633999999999		Meal/Entertain	6.0	82.07633999999999	Friday	Evening				
2016-01-06	17.0	15.0	2016-01-06	17.0	19.0	Jan	2016	Business	West Palm Beach	West Palm Beac
h 6.9201619999999995		Meal/Entertain	4.0	103.80242999999999	Wednesday	Afternoon				
2016-01-06	17.0	30.0	2016-01-06	17.0	35.0	Jan	2016	Business	West Palm Beach	Palm Beac
h 11.426314		Meeting	5.0	137.115768	Wednesday	Afternoon				
2016-01-07	13.0	27.0	2016-01-07	13.0	33.0	Jan	2016	Business	Cary	Car
y 1.2874720000000002		Meeting	6.0	12.874720000000002	Thursday	Afternoon				
2016-01-11	11.0	56.0	2016-01-11	12.0	3.0	Jan	2016	Business	Flatiron District	Midtow

Checking Category Wise Duration

```
# Category wise duration

df.groupBy("CATEGORY") \
  .agg(round(avg("duration"), 2)) \
  .show()
```

▶ (2) Spark Jobs

CATEGORY	round(avg(duration), 2)
Personal	20.47
Business	22.83

Top 10 Longest Trips

▶ Just now (1s)9Python

Top 10 Longest Trips

df.orderBy(df.duration.desc()).show(10)

▶ (1) Spark Jobs

start_date	start_hour	start_minute	end_date	end_hour	end_minute	month	year	CATEGORY	START	STO
P	MILES	PURPOSE	duration	speed	day	day_time				
2016-03-25	16.0	52.0	2016-03-25	22.0	22.0	Mar	2016	Business	Latta	Jacksonville
e	499.37820200000004	Customer Visit	330.0	90.79603672727274	Friday	Afternoon				
2016-07-14	16.0	39.0	2016-07-14	20.0	5.0	July	2016	Business	Morrisville	Banner El
k	314.304102	Errand/Supplies	206.0	91.5448840776699	Thursday	Afternoon				
2016-07-17	12.0	20.0	2016-07-17	15.0	25.0	July	2016	Personal	Boone	Car
y	290.003068	Commute	185.0	94.05504908108107	Sunday	Morning				
2016-10-30	15.0	22.0	2016-10-30	18.0	23.0	Oct	2016	Business	Asheville	Meban

Data Visualization:

▶ Just now (4s)10Python

import matplotlib.pyplot as plt

pdf = df.select("duration").toPandas()

plt.hist(pdf["duration"])

plt.title("Trip Duration Distribution")

plt.xlabel("Duration")

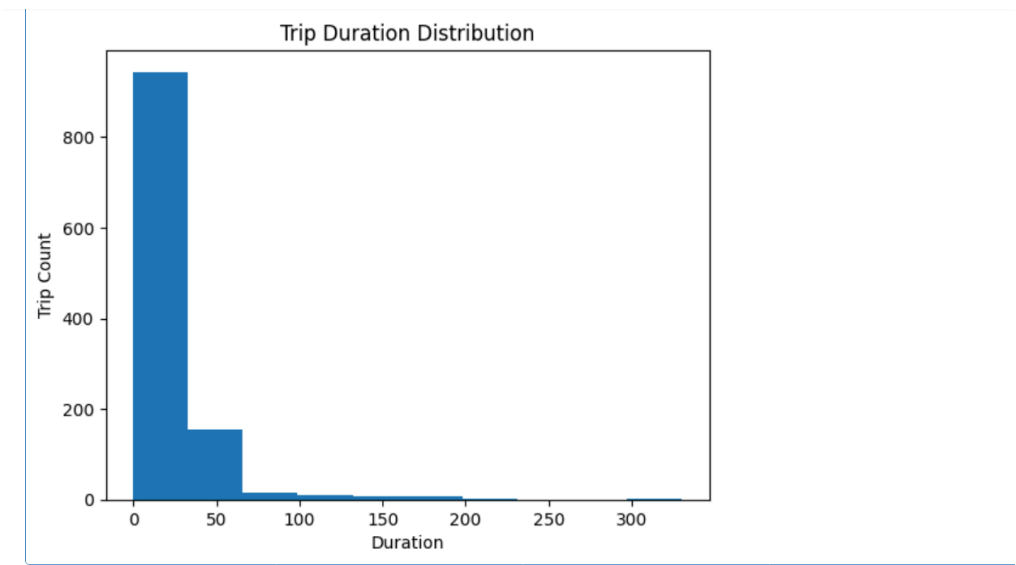
plt.ylabel("Trip Count")

plt.show()

▶ (1) Spark Jobs

> pdf: pandas.core.frame.DataFrame = [duration: float64]

Output:



4.2 Distance and Speed Analysis

Objective: Evaluate average speed per trip category, detect unrealistic speeds, and assess travel efficiency.

Benefit: Detects anomalies and monitors travel performance.

```
▶ ✓ 1 minute ago (<1s) 12
df = df.filter((df.duration > 0) & (df.MILES > 0))
> df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 15 more fields]
```

```
▶ ✓ 1 minute ago (<1s) 13
from pyspark.sql.functions import col

df = df.withColumn(
    "speed",
    col("MILES") / col("duration")
)
> df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 15 more fields]
```

```
▶ ✓ 1 minute ago (<1s) 14
df = df.filter(df.speed.isNotNull())
df = df.filter(df.speed != float("inf"))
> df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 15 more fields]
```

```
⋮ ▶ ✓ Just now (1s) 11 Python
from pyspark.sql.functions import avg, round
from pyspark.sql.functions import col

df = df.withColumn("speed", col("speed").cast("double"))
df.select(
    round(avg("speed"), 2)).alias("avg_speed").show()
▶ (2) Spark Jobs
> df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 15 more fields]
+-----+
|round(avg(speed), 2)|
+-----+
|          0.72|
+-----+
```

Finding low Speed

```
df.filter(df.speed < 5).show()
```

(1) Spark Jobs

start_date	start_hour	start_minute	end_date	end_hour	end_minute	month	year	CATEGORY	START	STO
P	MILES	PURPOSE	duration	speed	day	day_time				
2016-01-01	21.0	11.0	2016-01-01	21.0	17.0	Jan	2016	Business	Fort Pierce	Fort Pierc
e 8.207633999999999	Meal/Entertain	6.0	1.3679389999999998	Friday	Evening					
2016-01-02	1.0	25.0	2016-01-02	1.0	37.0	Jan	2016	Business	Fort Pierce	Fort Pierc
e	8.0467	Meal/Entertain	12.0	0.6705583333333333	Saturday	Night				
2016-01-02	20.0	25.0	2016-01-02	20.0	38.0	Jan	2016	Business	Fort Pierce	Fort Pierc
e 7.724831999999999	Errand/Supplies	13.0	0.5942178461538461	Saturday	Evening					
2016-01-05	17.0	31.0	2016-01-05	17.0	45.0	Jan	2016	Business	Fort Pierce	Fort Pierc
e	7.563898	Meeting	14.0	0.5402784285714286	Tuesday	Afternoon				
2016-01-06	14.0	42.0	2016-01-06	15.0	49.0	Jan	2016	Business	Fort Pierce	West Palm Beac
h 102.51495800000001	Customer Visit	67.0	1.5300740000000002	Wednesday	Afternoon					
2016-01-06	17.0	15.0	2016-01-06	17.0	19.0	Jan	2016	Business	West Palm Beach	West Palm Beac
h 6.9201619999999995	Meal/Entertain	4.0	1.7300404999999999	Wednesday	Afternoon					

Average Miles Covered:

```
df.select(
  | round(avg("MILES"),2).alias("avg_miles")
).show()

▶ (2) Spark Jobs
```

avg_miles
16.25

Data Visualization:

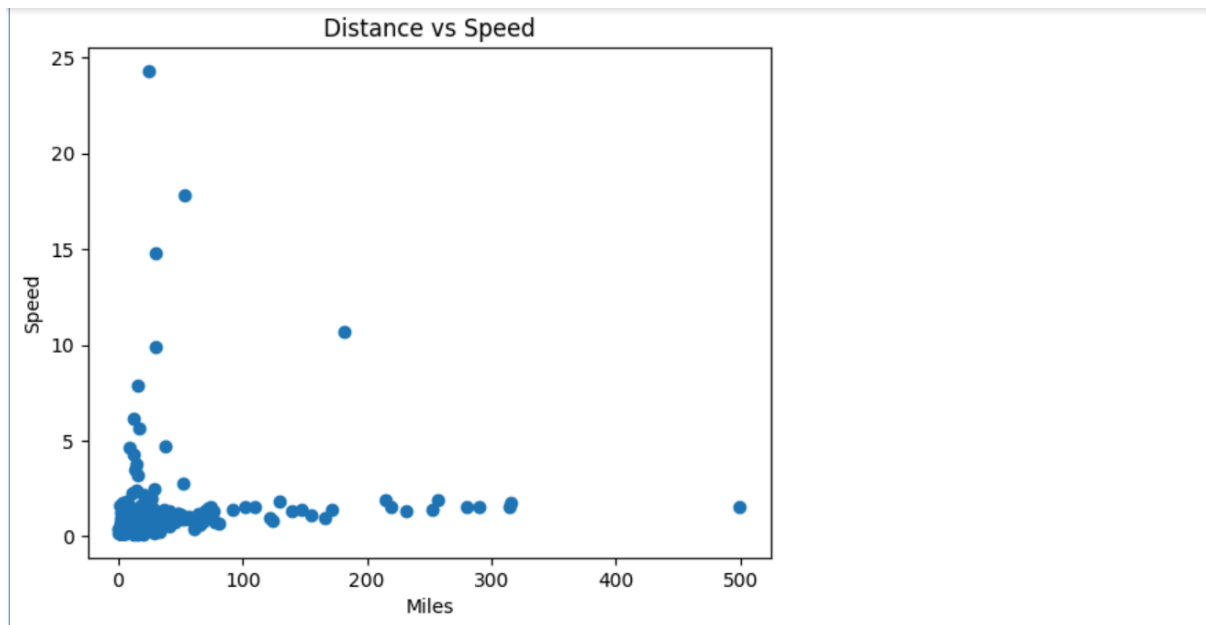
```
import matplotlib.pyplot as plt

pdf = df.select("MILES", "speed").toPandas()

plt.scatter(pdf["MILES"], pdf["speed"])
plt.xlabel("Miles")
plt.ylabel("Speed")
plt.title("Distance vs Speed")
plt.show()
```

(1) Spark Jobs

pdf: pandas.core.frame.DataFrame = [MILES: float64, speed: float64]



4.3 Time-of-Day Travel Patterns

Objective: Identify peak travel hours, analyze frequency by time period, and compare activity across day segments.

Benefit: Understands daily mobility patterns.

Checking the distinct values of day_time

```
df.select("day_time").distinct().show()
```

(2) Spark Jobs

day_time
Evening
Morning
Afternoon
Night

Counting the trp_count based on day_time

```
from pyspark.sql.functions import count

df.groupBy("day_time") \
    .agg(count("*").alias("trip_count")) \
    .orderBy("trip_count", ascending=False) \
    .show()
```

▸ (2) Spark Jobs

day_time	trip_count
Afternoon	555
Morning	312
Evening	233
Night	37

analysing peak travel time

```
# Peak travel period

df.groupBy("day_time") \
    .count() \
    .orderBy("count", ascending=False) \
    .limit(1) \
    .show()
```

▸ (2) Spark Jobs

day_time	count
Afternoon	555

Comparing category and time of the travel

```
# Category VS Time of the day

df.groupBy("day_time", "CATEGORY") \
    .count() \
    .orderBy("day_time") \
    .show()
```

▸ (2) Spark Jobs

day_time	CATEGORY	count
Afternoon	Business	523
Afternoon	Personal	32
Evening	Personal	15
Evening	Business	218
Morning	Business	286
Morning	Personal	26
Night	Personal	2
Night	Business	35

Calculating Average Duration per time Segment

```
Just now (1s) 24 Python
```

```
# Average Duration Per Time Segment

from pyspark.sql.functions import avg, round

df.groupBy("day_time").agg(
    round(avg("duration"),2).alias("avg_duration")
).show()
```

▶ (2) Spark Jobs

day_time	avg_duration
Evening	20.45
Morning	22.53
Afternoon	24.06
Night	19.7

Data Visualization:

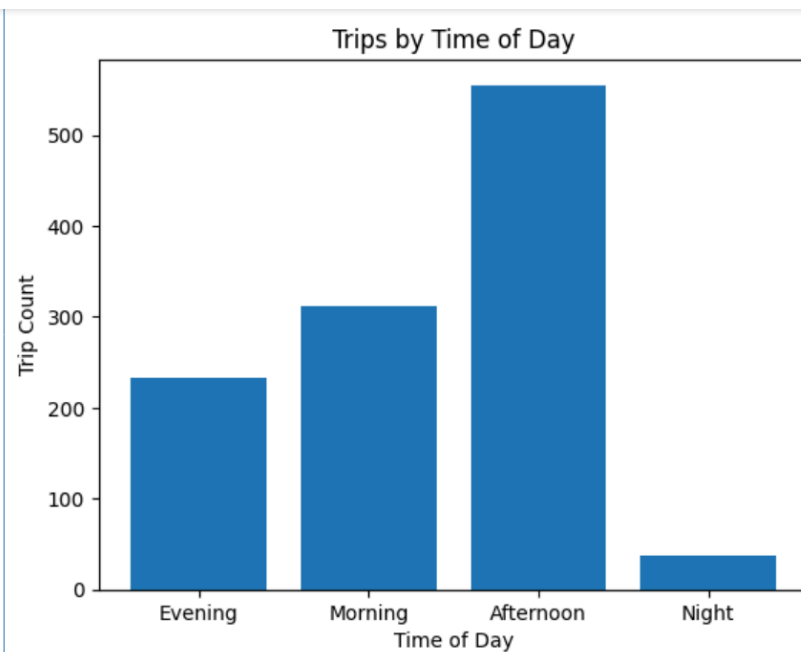
```
Just now (1s) 25 Python
```

```
import matplotlib.pyplot as plt

pdf = df.groupBy("day_time").count().toPandas()

plt.bar(pdf["day_time"], pdf["count"])
plt.xlabel("Time of Day")
plt.ylabel("Trip Count")
plt.title("Trips by Time of Day")
plt.show()
```

▶ (2) Spark Jobs



4.4 Day-wise Travel Patterns

Objective: Analyze travel across weekdays and weekends, identify busiest days, and calculate total distance per day.

Benefit: Supports planning and resource allocation.

Counting Trips Per day

```
Just now (1s) 28 Python
```

```
# Counting Trips Per Day

from pyspark.sql.functions import count

df.groupBy("day_of_week") \
  .agg(count("*").alias("trip_count")) \
  .orderBy("trip_count", ascending=False) \
  .show()
```

▸ (2) Spark Jobs

day_of_week	trip_count
Friday	201
Monday	174
Tuesday	172
Thursday	152
Sunday	147
Wednesday	146
Saturday	145

Calculating peak travel day based on week

```
Just now (1s) 29 Python
```

```
# Peak Travel Day

df.groupBy("day_of_week") \
  .count() \
  .orderBy("count", ascending=False) \
  .limit(1) \
  .show()
```

▸ (2) Spark Jobs

day_of_week	count
Friday	201

Average Duration Per Day

```
Just now (1s) 30 Python
```

```
# Average Duration Per Day

from pyspark.sql.functions import avg, round

df.groupBy("day_of_week").agg(
    round(avg("duration"),2).alias("avg_duration")
).show()
```

▶ (2) Spark Jobs

day_of_week	avg_duration
Wednesday	21.09
Tuesday	20.28
Friday	24.73
Thursday	24.65
Saturday	26.65
Monday	19.94
Sunday	22.17

Comparing Business and Personal by day

```
Just now (1s) 31 Python
```

```
# Business VS Personal by day

df.groupBy("day_of_week", "CATEGORY") \
    .count() \
    .orderBy("day_of_week") \
    .show()
```

▶ (2) Spark Jobs

day_of_week	CATEGORY	count
Friday	Business	187
Friday	Personal	14
Monday	Personal	9
Monday	Business	165
Saturday	Business	128
Saturday	Personal	17
Sunday	Business	141
Sunday	Personal	6
Thursday	Business	145
Thursday	Personal	7
Tuesday	Personal	13
Tuesday	Business	159
Wednesday	Personal	9
Wednesday	Business	137

Data Visualization:

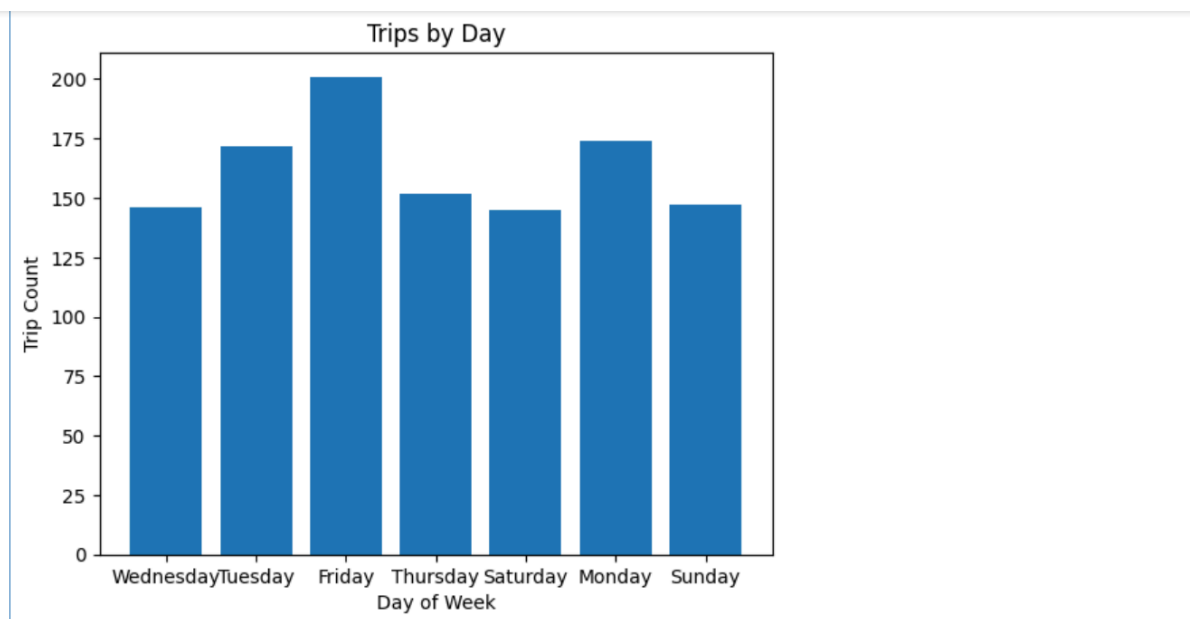
```
import matplotlib.pyplot as plt

pdf = df.groupby("day_of_week").count().toPandas()

plt.bar(pdf["day_of_week"], pdf["count"])
plt.xlabel("Day of Week")
plt.ylabel("Trip Count")
plt.title("Trips by Day")
plt.show()
```

▶ (2) Spark Jobs

> pdf: pandas.core.frame.DataFrame = [day_of_week: object, count: int64]



4.5 Category-Based Trip Analysis

Objective: Compare business vs. personal trips, analyze average distance and duration, and identify dominant travel purposes.

Benefit: Insights into travel behavior.

Calculating Trips per day

```
▶ ✓ Just now (1s) 33

# Trips per category

from pyspark.sql.functions import count

df.groupBy("CATEGORY") \
  .agg(count("*").alias("trip_count")) \
  .show()

▶ (2) Spark Jobs
```

CATEGORY	trip_count
Personal	75
Business	1062

Average Duration Per Category

```
Python ✕ ⌕ ⌕ ⋮

▶ ✓ Just now (1s) 34

# Average Duration Per Category

from pyspark.sql.functions import avg, round

df.groupBy("CATEGORY").agg(
  round(avg("duration"),2).alias("avg_duration")
).show()

▶ (2) Spark Jobs
```

CATEGORY	avg_duration
Personal	20.47
Business	22.92

Average Distance Per Category

```
▶ ✓ Just now (1s) 35

# Average Distance Per Category

df.groupBy("CATEGORY").agg(
  round(avg("MILES"),2).alias("avg_miles")
).show()

▶ (2) Spark Jobs
```

CATEGORY	avg_miles
Personal	15.16
Business	16.32

Comparing Category VS Day of week

```
Just now (1s) 37 Python
```

```
# Category VS Day of week

df.groupBy("CATEGORY","day_of_week") \
  .count() \
  .orderBy("CATEGORY") \
  .show()

▶ (2) Spark Jobs
```

CATEGORY	day_of_week	count
Business	Friday	187
Business	Wednesday	137
Business	Monday	165
Business	Sunday	141
Business	Saturday	128
Business	Thursday	145
Business	Tuesday	159
Personal	Saturday	17
Personal	Tuesday	13
Personal	Wednesday	9
Personal	Thursday	7
Personal	Friday	14
Personal	Monday	9
Personal	Sunday	6

Data Visualization:

```
1 minute ago (1s) 36 Python
```

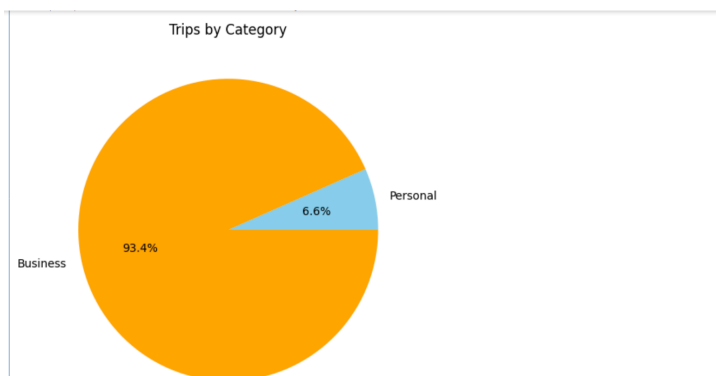
```
import matplotlib.pyplot as plt

pdf = df.groupBy("CATEGORY").count().toPandas()

plt.figure(figsize=(6,6))
plt.pie(
  pdf["count"],
  labels=pdf["CATEGORY"],
  autopct='%1.1f%%',
  colors=['skyblue','orange']
)

plt.title("Trips by Category")
plt.show()

▶ (2) Spark Jobs
```



4.6 Monthly Travel Trends

Objective: Track total trips per month and identify seasonal trends across years.

Benefit: Helps with seasonal demand planning.

```
▶ Just now (1s) 38 Python
```

```
# Trips per Month

from pyspark.sql.functions import count, asc

df.groupBy("month") \
  .agg(count("*").alias("trip_count")) \
  .orderBy(asc("trip_count")) \
  .show()
```

▶ (2) Spark Jobs

month	trip_count
Sep	33
May	48
April	53
Jan	61
Oct	103
June	103
July	109
Mar	112
Feb	114
Nov	122
Aug	133
Dec	146

```
▶ Just now (1s) 39 Python
```

```
# Average Duration Per Month

from pyspark.sql.functions import avg, round

df.groupBy("month").agg(
  round(avg("duration"),2).alias("avg_duration")
).orderBy("avg_duration").show()
```

▶ (2) Spark Jobs

month	avg_duration
May	14.65
Nov	17.68
Jan	18.3
Feb	19.73
June	20.11
Dec	21.37
July	21.55
Mar	24.5
Aug	24.98
April	29.81
Oct	30.48
Sep	40.18

```
df.groupBy("month") \
  .count() \
  .orderBy("count", ascending=False) \
  .limit(1) \
  .show()
```

▶ (2) Spark Jobs

month	count
Dec	146

Data Visualization:

```
import matplotlib.pyplot as plt

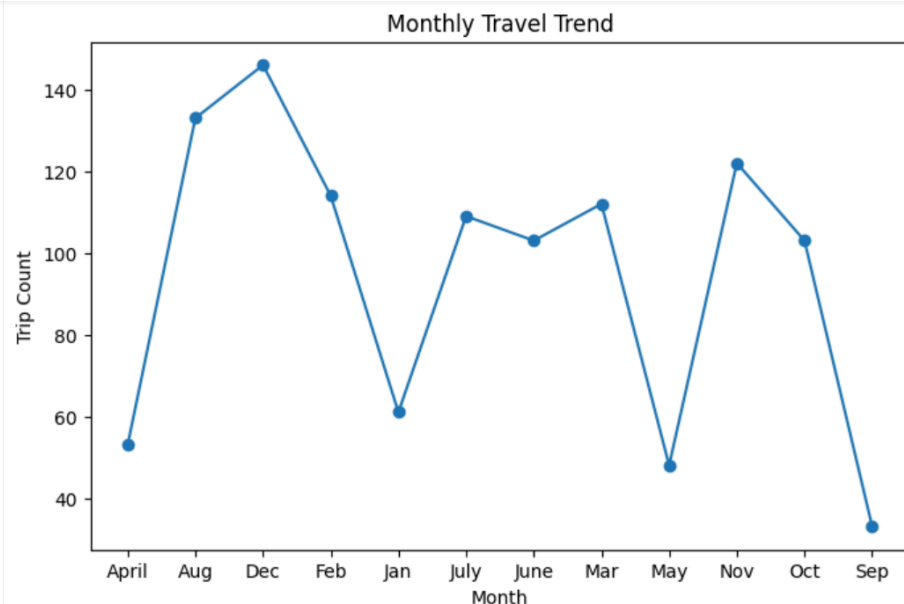
pdf = df.groupBy("month").count().orderBy("month").toPandas()

plt.figure(figsize=(8,5))
plt.plot(pdf["month"], pdf["count"], marker='o')

plt.xlabel("Month")
plt.ylabel("Trip Count")
plt.title("Monthly Travel Trend")
plt.show()
```

▶ (4) Spark Jobs

pdf: pandas.core.frame.DataFrame = [month: object, count: int64]



4.7 Travel Efficiency Analysis

Metrics: Miles per trip, duration per mile, average speed.

Benefit: Highlights inefficiencies and potential improvements.

```
▶ ✓ Just now (<1s) 42

# Creating Distance Bucket

from pyspark.sql.functions import when, col

df = df.withColumn(
    "distance_bucket",
    when(col("MILES") < 5, "Short Trip")
    .when((col("MILES") >= 5) & (col("MILES") < 20), "Medium Trip")
    .otherwise("Long Trip")
)

> df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 17 more fields]
```

```
▶ ✓ Just now (1s) 44 Python

# Average Duration Per Bucket

from pyspark.sql.functions import avg, round

df.groupBy("distance_bucket").agg(
    round(avg("duration"),2).alias("avg_duration")
).show()

▶ (2) Spark Jobs
```

distance_bucket	avg_duration
Medium Trip	20.2
Short Trip	9.03
Long Trip	50.92

Data visualization:

```
▶ ✓ Just now (1s) 45 Python

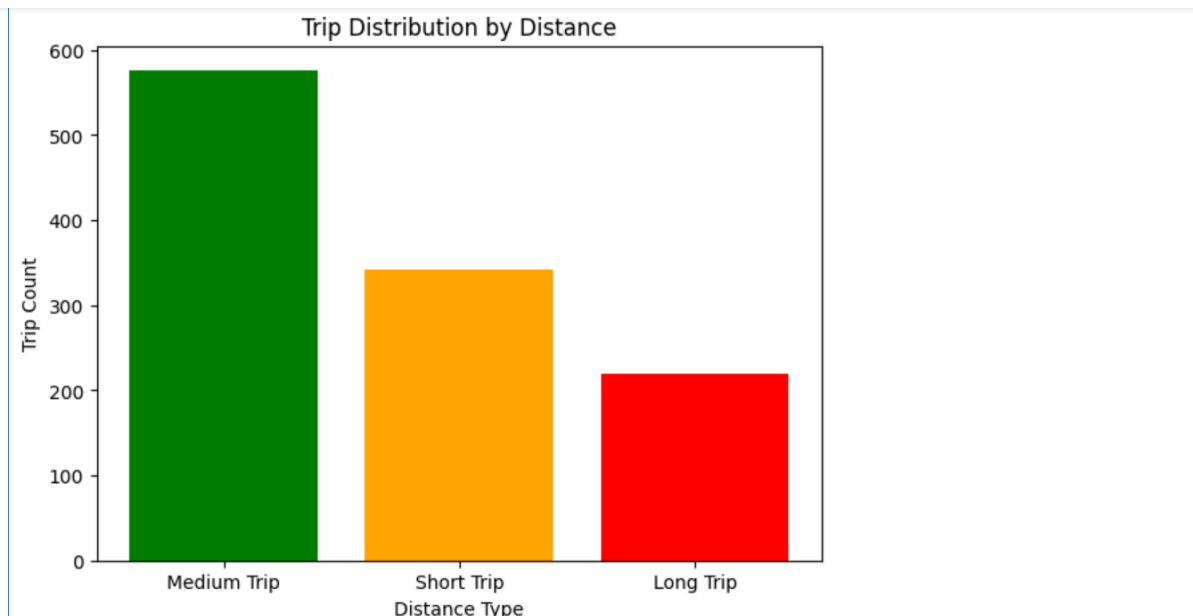
import matplotlib.pyplot as plt

pdf = df.groupBy("distance_bucket").count().toPandas()

plt.figure(figsize=(7,5))
plt.bar(pdf["distance_bucket"], pdf["count"], color=['green', 'orange', 'red'])

plt.xlabel("Distance Type")
plt.ylabel("Trip Count")
plt.title("Trip Distribution by Distance")
plt.show()

▶ (2) Spark Jobs
```



4.8 Peak Travel Period Detection

Objective: Count trips per hour, identify peak windows, and analyze time distribution.

Benefit: Supports demand forecasting and traffic management.

```
▶ ✓ 1 minute ago (<1s) 46

# Create Weekend Column

from pyspark.sql.functions import when, col

df = df.withColumn(
    "day_type",
    when(col("day_of_week").isin("Saturday", "Sunday"), "Weekend")
    .otherwise("Weekday")
)

> df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 18 more fields]
```

```
▶ ✓ 1 minute ago (1s) 47

# Count Trips

df.groupBy("day_type") \
  .count() \
  .show()

▶ (2) Spark Jobs

+-----+-----+
|day_type|count|
+-----+-----+
| Weekday|  845|
| Weekend|  292|
+-----+-----+
```

▶

✓ 1 minute ago (1s)

48

Average Duration

from pyspark.sql.functions import avg, round

df.groupBy("day_type").agg(
| round(avg("duration"),2).alias("avg_duration")
|).show()

▶ (2) Spark Jobs

+-----+-----+
|day_type|avg_duration|
+-----+-----+
| Weekday| 22.19|
| Weekend| 24.39|
+-----+-----+

▶

✓ 1 minute ago (1s)

49

Average Distance

df.groupBy("day_type").agg(
| round(avg("MILES"),2).alias("avg_distance")
|).show()

▶ (2) Spark Jobs

+-----+-----+
|day_type|avg_distance|
+-----+-----+
| Weekday| 15.21|
| Weekend| 19.25|
+-----+-----+

Data Visualization:

⋮

▶

✓ Just now (1s)

50

Python

🗑️

✎

🔍

⋮

import matplotlib.pyplot as plt

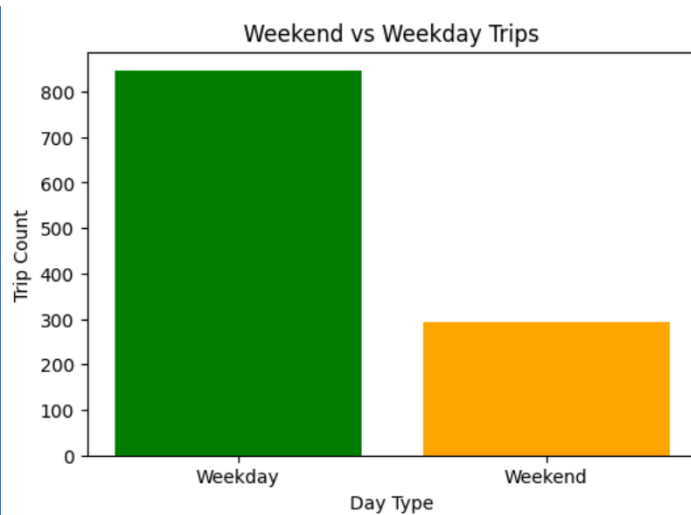
pdf = df.groupBy("day_type").count().toPandas()

plt.figure(figsize=(6,4))
plt.bar(pdf["day_type"], pdf["count"], color=['green','orange'])

plt.xlabel("Day Type")
plt.ylabel("Trip Count")
plt.title("Weekend vs Weekday Trips")
plt.show()

▶ (2) Spark Jobs

> pdf: pandas.core.frame.DataFrame = [day_type: object, count: int64]



4.9 Long Distance Trip Identification

Objective: Detect trips exceeding distance thresholds or unusually long durations.

Benefit: Evaluates travel costs and logistics.

```
▶ ✓ 2 minutes ago (1s) 51

# Top Start Locations

from pyspark.sql.functions import count

df.groupBy("START") \
  .agg(count("*").alias("trip_count")) \
  .orderBy("trip_count", ascending=False) \
  .show(10)
```

▶ (2) Spark Jobs

```
+-----+-----+
|      START|trip_count|
+-----+-----+
|      Cary|      201|
|Unknown Location|      144|
|    Morrisville|       84|
|    Whitebridge|       68|
|    Islamabad|       56|
|      Lahore|       36|
|      Durham|       30|
|    Raleigh|       28|
```



```
▶ ✓ 2 minutes ago (1s) 52

# Top End Locations

df.groupBy("STOP") \
  .agg(count("*").alias("trip_count")) \
  .orderBy("trip_count", ascending=False) \
  .show(10)

▶ (2) Spark Jobs
```

STOP	trip_count
Cary	195
Unknown Location	146
Morrisville	84
Whitebridge	65
Islamabad	57
Lahore	36
Durham	36
Raleigh	29
Kar?chi	26
Apex	17

```
▶ ✓ 1 minute ago (1s) 53

# Most Common Routes

df.groupBy("START", "STOP") \
  .count() \
  .orderBy("count", ascending=False) \
  .show(10)

▶ (2) Spark Jobs
```

START	STOP	count
Unknown Location	Unknown Location	83
Morrisville	Cary	74
Cary	Morrisville	67
Cary	Cary	53
Cary	Durham	36
Unknown Location	Islamabad	28
Islamabad	Unknown Location	28
Lahore	Lahore	27
Durham	Cary	25
Islamabad	Islamabad	24

Data Visualization:

```
⋮ ▶ ✓ 1 minute ago (1s) 54 Python 🗑️ ✨ 📐 ⋮

import matplotlib.pyplot as plt

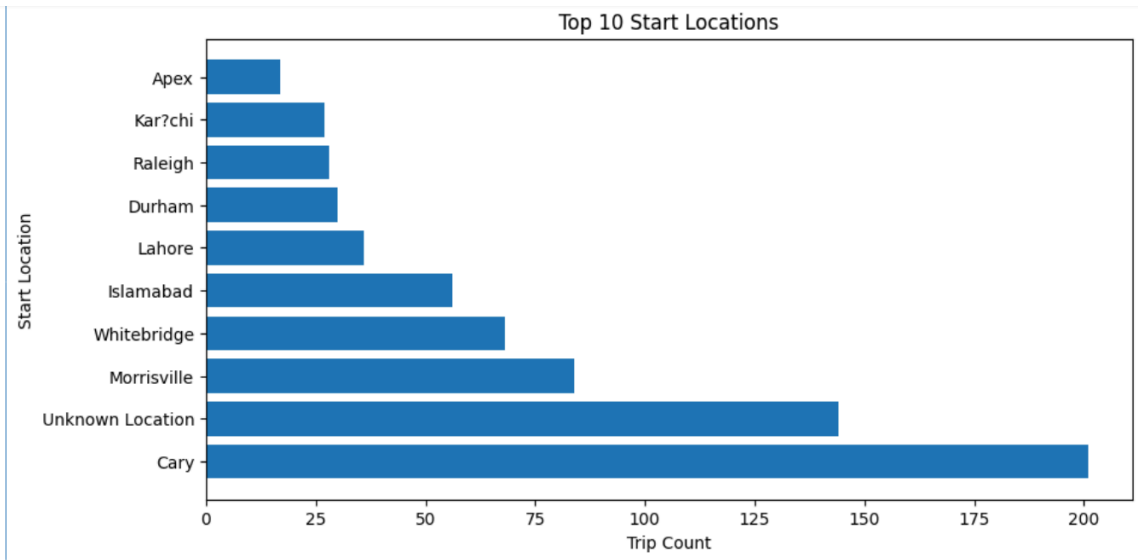
pdf = df.groupBy("START") \
  .count() \
  .orderBy("count", ascending=False) \
  .limit(10) \
  .toPandas()

plt.figure(figsize=(10,5))
plt.barh(pdf["START"], pdf["count"])

plt.xlabel("Trip Count")
plt.ylabel("Start Location")
plt.title("Top 10 Start Locations")
plt.show()

▶ (2) Spark Jobs

> pdf: pandas.core.frame.DataFrame = [START: object, count: int64]
```



4.10 Data Quality Monitoring

Possible Issues: Negative durations, missing values, unrealistic speeds, zero distance with long duration, start time after end time.

Benefit: Ensures clean and reliable data for analysis.

```
1 minute ago (<1s) 55

# Recalculating Speed to avoid infinity

from pyspark.sql.functions import col

df = df.filter((col("duration") > 0) & (col("MILES") > 0))

df = df.withColumn(
    "speed",
    col("MILES") / col("duration")
)

> df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 18 more fields]
```

```
Just now (<1s) 57

# Speed Bucket

from pyspark.sql.functions import when

df = df.withColumn(
    "speed_bucket",
    when(col("speed") < 0.5, "Slow")
    .when((col("speed") >= 0.5) & (col("speed") < 1.0), "Normal")
    .otherwise("Fast")
)

> df: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 19 more fields]
```

```
Just now (1s) 58

# Count Trips per Speed Type

df.groupBy("speed_bucket") \
  .count() \
  .show()

▶ (2) Spark Jobs
```

speed_bucket	count
Fast	129
Slow	461
Normal	547

Data Visualization:

```
Just now (2s) 59 Python
```

```
import matplotlib.pyplot as plt

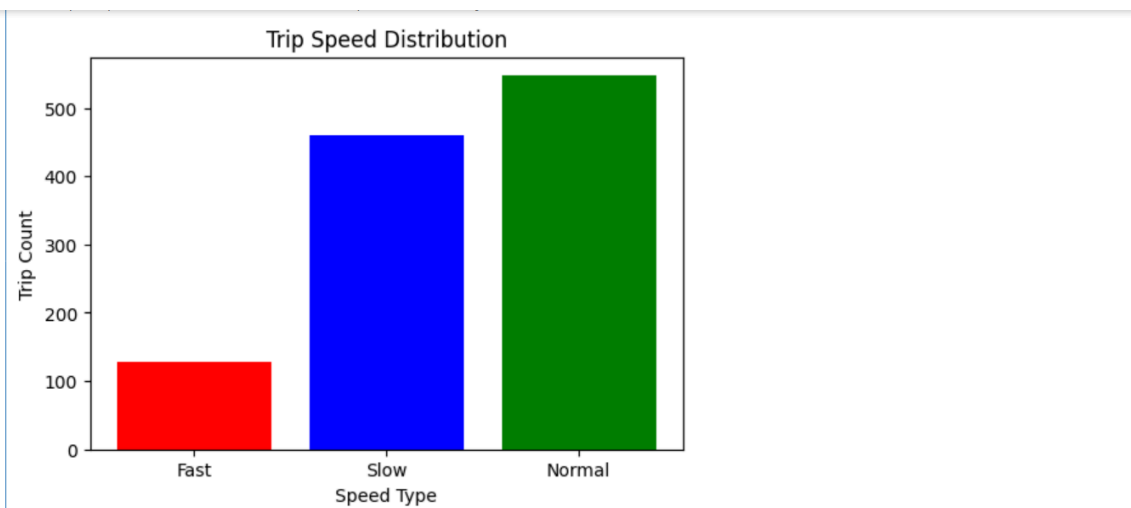
pdf = df.groupBy("speed_bucket").count().toPandas()

plt.figure(figsize=(6,4))
plt.bar(pdf["speed_bucket"], pdf["count"], color=['red','blue','green'])

plt.xlabel("Speed Type")
plt.ylabel("Trip Count")
plt.title("Trip Speed Distribution")
plt.show()

▶ (2) Spark Jobs
```

> pdf: pandas.core.frame.DataFrame = [speed_bucket: object, count: int64]



4.11 Travel Behavior Segmentation

Objective: Use clustering algorithms to identify groups of similar trips based on distance, duration, speed, and time of day.

Benefit: Reveals distinct travel behaviors.

✓ 1 minute ago (1s)

61

Trips Per Purpose

```
from pyspark.sql.functions import count

df.groupBy("PURPOSE") \
  .agg(count("*").alias("trip_count")) \
  .orderBy("trip_count", ascending=False) \
  .show()
```

▶ (2) Spark Jobs

PURPOSE	trip_count
Meeting	295
Meal/Entertain	245
Temporary Site	191
Errand/Supplies	173
Customer Visit	155
Airport/Travel	54
Between Offices	18
Moving	4
Charity (\$)	1

✓ 1 minute ago (1s)

62

Average Duration Per Purpose

```
from pyspark.sql.functions import avg, round

df.groupBy("PURPOSE").agg(
  round(avg("duration"),2).alias("avg_duration")
).show()
```

▶ (2) Spark Jobs

PURPOSE	avg_duration
Airport/Travel	34.93
Charity (\$)	27.0
Customer Visit	28.39
Temporary Site	25.42
Errand/Supplies	15.71
Between Offices	25.5
Commute	185.0
Meal/Entertain	15.93
Moving	15.0
Meeting	25.03

✓ 1 minute ago (1s)

63

Average Duration Per Purpose

```
df.groupBy("PURPOSE").agg(
  round(avg("MILES"),2).alias("avg_miles")
).show()
```

▶ (2) Spark Jobs

PURPOSE	avg_miles
Airport/Travel	24.07
Charity (\$)	24.3
Customer Visit	25.86
Temporary Site	15.85
Errand/Supplies	9.54
Between Offices	17.61
Commute	290.0
Meal/Entertain	9.29
Moving	7.32
Meeting	18.82

Data Visualization:

```
import matplotlib.pyplot as plt

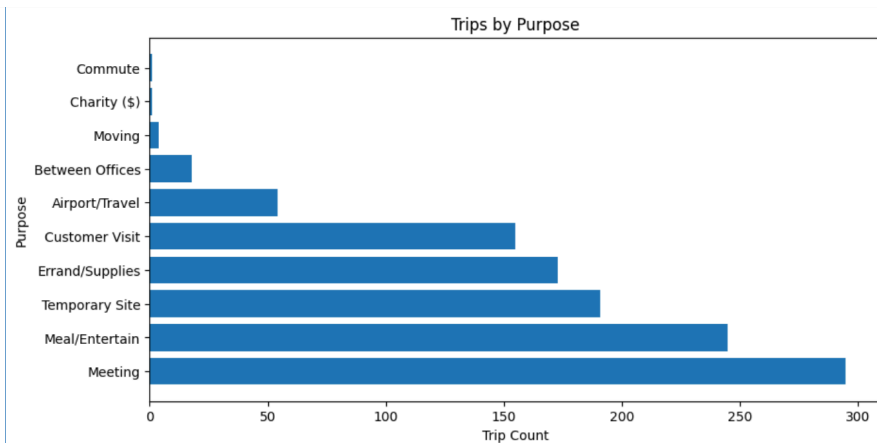
pdf = df.groupBy("PURPOSE") \
    .count() \
    .orderBy("count", ascending=False) \
    .limit(10) \
    .toPandas()

plt.figure(figsize=(10,5))
plt.barh(pdf["PURPOSE"], pdf["count"])

plt.xlabel("Trip Count")
plt.ylabel("Purpose")
plt.title("Trips by Purpose")
plt.show()
```

▶ (2) Spark Jobs

▶ pdf: pandas.core.frame.DataFrame = [PURPOSE: object, count: int64]



4.12 Travel Duration Prediction

Objective: Train regression models to predict travel time using distance, time of day, day of week, and trip category.

Benefit: Improves planning and estimation.

```
# Trips Per Hour

from pyspark.sql.functions import count

df.groupBy("start_hour") \
    .agg(count("*").alias("trip_count")) \
    .orderBy("start_hour") \
    .show(24)
```

▶ (2) Spark Jobs

start_hour	trip_count
0.0	19
1.0	5
2.0	2
3.0	3
5.0	4
6.0	4
7.0	12
8.0	35

```
Just now (1s) 66

# Calculating Peak Hour

df.groupBy("start_hour") \
  .count() \
  .orderBy("count", ascending=False) \
  .limit(1) \
  .show()

(2) Spark Jobs
```

start_hour	count
15.0	97

Data Visualization

```
Just now (1s) 67 Python
```

```
import matplotlib.pyplot as plt

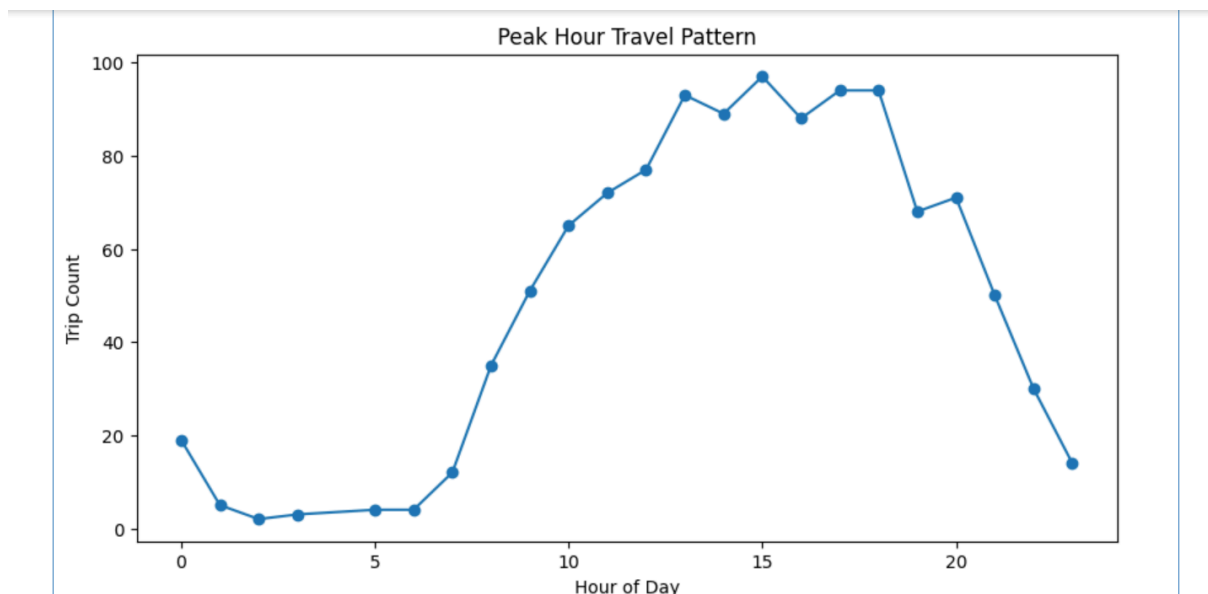
pdf = df.groupBy("start_hour").count().orderBy("start_hour").toPandas()

plt.figure(figsize=(10,5))
plt.plot(pdf["start_hour"], pdf["count"], marker='o')

plt.xlabel("Hour of Day")
plt.ylabel("Trip Count")
plt.title("Peak Hour Travel Pattern")
plt.show()

(4) Spark Jobs
```

> pdf: pandas.core.frame.DataFrame = [start_hour: float64, count: int64]



5. Tools and Technologies

- Microsoft Azure
- Azure Data Lake Storage
- Azure Databricks
- Apache Spark / PySpark
- Python
- Matplotlib, Seaborn

6. Visualization and Reporting

After processing with PySpark, aggregated results are converted to **Pandas DataFrames** for visualization:

- **Trip Frequency by Hour:** Shows trips per hour.
- **Distance Distribution:** Shows how distances vary.
- **Distance vs Duration Scatter Plot:** Reveals travel efficiency and anomalies.
- **Trips by Category:** Compares business vs. personal trips.
- **Trips by Day:** Highlights daily patterns.

These visualizations help interpret and communicate insights effectively.

7. Conclusion

Using **PySpark** on **Azure**, the system efficiently processes large trip datasets and provides actionable insights. Integration of distributed computing, cloud storage, and visualization tools supports operational improvements and strategic decision-making.